



INSTITUTO POLITÉCNICO

CURSO DE ENGENHARIA INFORMÁTICA

RELATÓRIO DO PROJETO DE S.G.B.D

Integrantes do grupo:

- 1. Simão Kiyeleko**
- 2. João Fernando da Costa**
- 3. Maria David Kiala**
- 4. João Pedro Lucunga**
- 5. Lukoki Luvualo**
- 6. Augusto Paulo**

INTRODUÇÃO

No contexto tecnológico atual, a evolução dos sistemas de comércio eletrônico, exige soluções de armazenamento de dados altamente escaláveis, flexíveis e de excelente desempenho. Os modelos relacionais tradicionais, embora robustos, muitas vezes enfrentam limitações ao lidar com grandes volumes de dados não estruturados ou com catálogos de produtos de esquemas dinâmicos.

Para suprir estas necessidades, este projeto foca-se na concepção e implementação de uma infraestrutura moderna de gestão de dados baseada em tecnologia NoSQL.

OBJETIVO GERAL

O objetivo geral deste trabalho consiste em projetar, configurar e validar uma infraestrutura automatizada e portátil utilizando uma base de dados NoSQL orientada a documentos (MongoDB), integrada num ambiente de contentores (Docker) sob o ecossistema Windows Subsystem for Linux.

IMPORTÂNCIA DA FERRAMENTA

A seleção do MongoDB como sistema de gestão de base de dados NoSQL e do Docker como plataforma de contentorização reveste-se de extrema importância estratégica e técnica pelas seguintes razões:

Flexibilidade de esquema (Schema-less): Ao contrário das bases de dados SQL, o MongoDB armazena dados em documentos BSON (JSON binário), o que permite modificar a estrutura de um produto (adicionar atributos, cores ou especificações técnicas) sem a necessidade de migrações complexas ou interrupções no serviço.

Desempenho e escalabilidade horizontal: O MongoDB foi nativamente desenhado para oferecer alta disponibilidade e escalabilidade através de réplicas e sharding, permitindo que o sistema de e-commerce cresça em volume de utilizadores e transações mantendo tempos de resposta reduzidos.

Isolamento e Portabilidade com Docker: A utilização de contentores garante que toda a infraestrutura da base de dados e as suas configurações específicas corram de forma isolada do sistema operativo hospedeiro. Isto elimina o clássico problema de compatibilidade ('funciona na minha máquina') e facilita a replicação imediata do ambiente em servidores de produção ou nuvem.

Eficiência no Desenvolvimento: O ecossistema permite acoplar facilmente scripts em Python para automação, testes de carga massivos e auditoria de logs, acelerando o ciclo de entrega de software.

INFRAESTRUTURA E VIRTUALIZAÇÃO .

Com o objetivo de criar um ambiente estável e idêntico ao de produção, a camada de infraestrutura foi implementada da seguinte forma:

Sistema Hospedeiro e WSL: configuração do windows subsystem for Linux (WSL) sobre o windows 10 para fornecer suporte nativo a ferramentas Linux.

```
Selecionar Linha de comandos - wsl --update
Microsoft Windows [Version 10.0.19045.7291]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\msima>wsl --update
Installing: Windows Subsystem for Linux
[=====50,0%]
```

Após a atualização do WSL, foi consultado o terminal Ubuntu 24.04.1 LTS. O ambiente Linux confirmou a conectividade de rede e os recursos do sistema, estando o ambiente pronto para suportar o Docker e as ferramentas de desenvolvimento.

```
simao@DESKTOP-157I6QL: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 4.4.0-19041-Microsoft x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Wed May 27 16:46:45 WAT 2026

System load:          0.52
Usage of /home:       unknown
Memory usage:         88%
Swap usage:           3%
Processes:            8
Users logged in:      0
IPv4 address for wif0: 192.168.0.222
IPv6 address for wif0: 2c0f:f888:a180:6e46:2beb:4855:bd18:9a17
IPv6 address for wif0: 2c0f:f888:a180:6e46:cddc:6552:eb4:8bb8

=> /mnt/c is using 98.0% of 106.76GB

This message is shown once a day. To disable it please create the
/home/simao/.hushlogin file.
simao@DESKTOP-157I6QL:~$ ls
simao@DESKTOP-157I6QL:~$ ls -l
total 0
simao@DESKTOP-157I6QL:~$ cd
simao@DESKTOP-157I6QL:~$
```

Com o WSL funcional, procedeu-se à criação e execução do contentor Docker. O contentor foi nomeado como mongodb-e-commerce, com a porta padrão 27017 mapeada, credenciais administrativas definidas, volume de persistência configurado em D:\docker\mongodb\data e política de reinicialização automática unless-stopped ativada.

```
PS C:\Users\msima> docker run -d `
>> --name mongodb-e-commerce `
>> -p 27017:27017 `
>> -e MONGO_INITDB_ROOT_USERNAME=admin `
>> -v D:\docker\mongodb\data:/data/db `
>> --restart unless-stopped `
>> mongo
5f15ac8025210721554b5f7b0d2f8c269232dc2e712b8642192d8528744de33a
PS C:\Users\msima> docker ps
Using MongoDB:      undefined
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

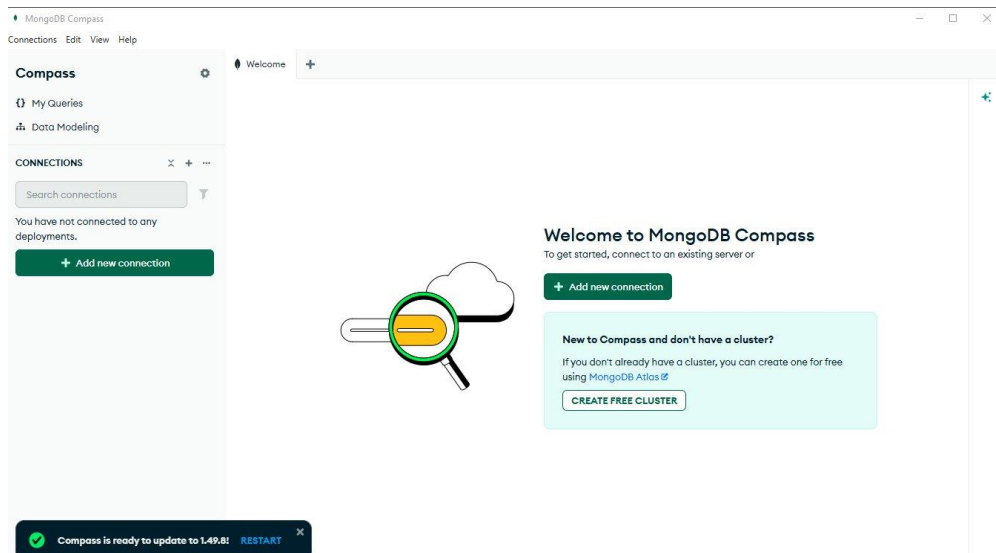
To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.
```

ADMINISTRAÇÃO E CONECTIVIDADE

Para os processos de monitorização e validação dos dados estruturados, foi configurado o cliente oficial MongoDB Compass:

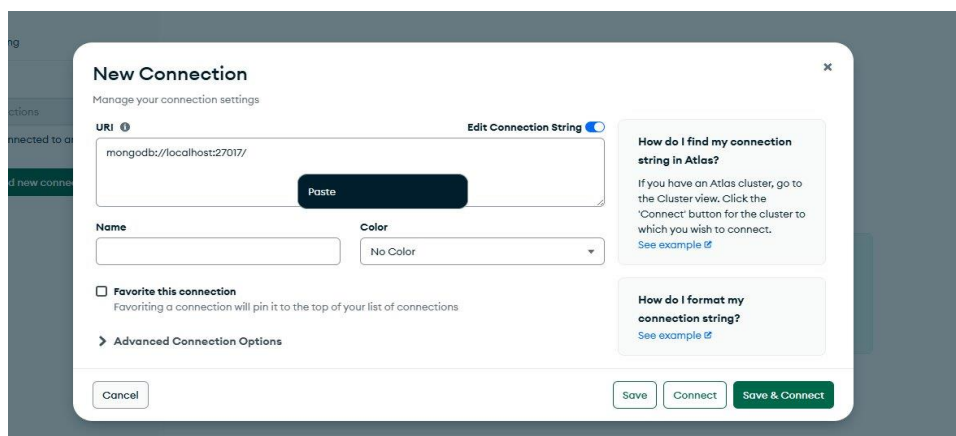
Abertura do MongoDB compass

Após a instalação, o MongoDB Compass foi iniciado apresentando o ecrã de boas-vindas, pronto para estabelecer uma nova ligação ao servidor local. O cliente informou também sobre uma atualização disponível para a versão 1.49.8.

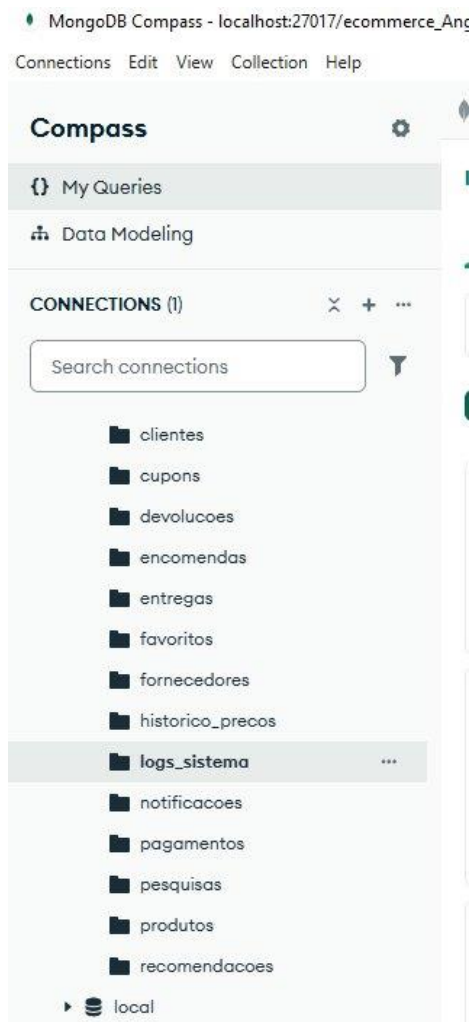


CONFIGURAÇÃO DO STRING DE CONEXÃO

A ligação ao servidor foi estabelecida através da string de conexão padrão `mongodb://localhost:27017/`, apontando para o contentor Docker em execução na porta local 27017. A opção Edit Connection String foi ativada para configuração manual do URI.

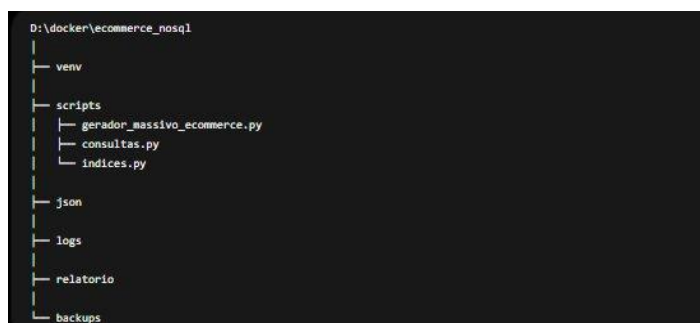


Após a conexão bem-sucedida, o Compass exibiu todas as coleções da base de dados `ecommerce_catalog`. As coleções abrangem as diferentes entidades do sistema de e-commerce, incluindo clientes, produtos, encomendas, pagamentos, entregas, fornecedores, entre outras.



A organização do diretório raiz do projeto (D:\docker\ecommerce_nosql) garante a modularidade e manutenibilidade do sistema. A estrutura adotada separa claramente os ambientes virtuais, scripts de automação, dados JSON, logs e backups.

ESQUEMA DE DIRETÓRIOS DO PROJECTO



ARQUITETURA DE PASTAS E AUTOMAÇÃO DE SCRIPTS

A organização do diretório raiz do projeto ('D:\docker\ecommerce_nosql') garante a modularidade do sistema. A tabela seguinte descreve a função técnica de cada componente da estrutura de pastas:

Diretório / Ficheiro	Descrição e Função Técnica
venv/	Ambiente virtual Python isolado contendo as dependências de pacotes (ex: pymongo).
scripts/gerador_massivo_ecommerce.py	Script responsável pela geração e injeção em massa de dados fictícios para testes de carga e stress.
scripts/consultas.py	Script que centraliza a lógica das queries, agregações e filtros de pesquisa de negócio.
scripts/indices.py	Script focado na criação e manutenção de índices para otimização da performance de busca.
json/ , logs/ , relatorio/ , backups/	Pastas utilitárias destinadas a esquemas de dados, logs de erro, relatórios técnicos e cópias de segurança (backups).

EXECUÇÃO DOS SCRIPTS

O script `gerador_massivo_ecommerce.py` foi executado com sucesso, conectando-se ao MongoDB e gerando os documentos em massa. A imagem abaixo demonstra a barra de progresso durante a geração de 500 000 registos de clientes (concluídos em 200.7s a 2491 docs/s) e o início da geração de 300 000 produtos.

```

2026-06-01 18:56:03,001 [INFO]  Ligação ao MongoDB estabelecida com sucesso.
2026-06-01 18:56:03,017 [INFO]  Gerando clientes...
Clientes: 100% | 500000/500000 [03:20<00:00, 2492.98doc/s]
2026-06-01 18:59:23,734 [INFO]  Clientes concluídos em 200.7s (2491 docs/s)
2026-06-01 18:59:23,747 [INFO]  Gerando produtos...
Produtos: 12% | 35000/300000 [00:08<01:26, 3075.39doc/s]

```

CONCLUSÕES

A integração do ecossistema NoSQL estruturado neste projeto provou ser uma escolha altamente eficaz. O isolamento em Docker oferece um ciclo de desenvolvimento limpo e livre de conflitos de dependências, enquanto o MongoDB assegura que a plataforma de E-Commerce esteja preparada para escalabilidade futura, mantendo uma experiência fluida através do uso otimizado de índices de pesquisa.